ITI – Analytical SQL lab2

Mohammad Tarek Omar – Data Management

Q1:

```sql
-- 1) Identify The Top 3 employees in Each Department and their revenue based on their
-- revenue_generated from employee_performance

select distinct
    department,
    nth_value(emp_name, 1) over(partition by department order by revenue_generated desc rows between unbounded preceding and unbounded following) as top_1_emp,
    nth_value(revenue_generated, 1) over(partition by department order by revenue_generated desc rows between unbounded preceding and unbounded following) as top_1_rev,
    nth_value(emp_name, 2) over(partition by department order by revenue_generated desc rows between unbounded preceding and unbounded following) as top_2_emp,
    nth_value(revenue_generated, 2) over(partition by department order by revenue_generated desc rows between unbounded preceding and unbounded following) as top_2_rev,
    nth_value(emp_name, 3) over(partition by department order by revenue_generated desc rows between unbounded preceding and unbounded following) as top_3_emp,
    nth_value(revenue_generated, 3) over(partition by department order by revenue_generated desc rows between unbounded preceding and unbounded following) as top_3_rev
from employee_performance
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department	top_1_emp	top_1_rev	top_2_emp	top_2_rev	top_3_emp	top_3_rev
Finance	Karen	42000	Jack	40000	Leo	38000
HR	Hank	12000	Steve	11000	Grace	10000
IT	Rachel	33000	Nina	31000	Eve	30000
Sales	Quinn	24000	Charlie	22000	Mia	21000

Q2:

```sql
-- 2) Calculate Year to Date Revenue for Employee Quinn only, Year to date is the sum of the
-- revenue starting from the start of current year, till the date of calculation.

select emp_name,
    extract(year from evaluation_date) as year,
    revenue_generated,
    sum(revenue_generated) over(partition by year(evaluation_date) order by evaluation_date) as year_to_date_revenue
from employee_performance
where emp_name = 'quinn'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	year	revenue_generated	year_to_date_revenue
Quinn	2023	15000	15000
Quinn	2024	24000	24000
Quinn	2024	12000	36000

Q3:

```sql
-- 3) Identify Departments with the Most Evenly Distributed Revenue , comment on your
-- answer.

select distinct
    department,
    round(stddev(revenue_generated) over(partition by department), 2) as rev_stddev
from employee_performance
order by rev_stddev
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department	rev_stddev
HR	1414.21
Finance	2561.25
IT	2856.57
Sales	3870.35

Q4:

```sql
-- 4) Calculate the moving Average working hours  for Emp Quinn for last 5 rows.

select
    emp_name,
    evaluation_date,
    hours_worked,
    avg(hours_worked) over(order by evaluation_date rows between 4 preceding and current row) as moving_avg_hours
from employee_performance
where emp_name = 'quinn'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	evaluation_date	hours_worked	moving_avg_hours
Quinn	2023-03-30	100	100.0000
Quinn	2024-03-30	140	120.0000
Quinn	2024-04-30	120	120.0000

Q5:

```sql
-- 5) Compare Revenue with a 2-Month Range
-- For each evaluation, calculate the total revenue for evaluations that occurred within 1
-- month (60 days) before and including the current date.

select
    emp_name,
    evaluation_date,
    revenue_generated,
    sum(revenue_generated) over(order by evaluation_date range between interval '1' month preceding and current row) as moving_range_rev
from employee_performance
where emp_name = 'quinn'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	evaluation_date	revenue_generated	moving_range_rev
Quinn	2023-03-30	15000	15000
Quinn	2024-03-30	24000	24000
Quinn	2024-04-30	12000	36000

Q6:

```sql
-- 6) Flag Employees Who Are Falling Behind Department's Rolling Average
-- Business Need:
-- Identify employees whose revenue is consistently below their department's rolling average
-- over the last 3 rows.

with RollingAvgData as (select emp_name, department, evaluation_date, revenue_generated,
    round(avg(revenue_generated) over (partition by department order by evaluation_date rows between 2 preceding and current row), 2) as rolling_av
    from employee_performance
)
select
    emp_name,
    department,
    evaluation_date,
    revenue_generated,
    rolling_avg_department_revenue,
    case when revenue_generated < rolling_avg_department_revenue then 'Below Rolling Avg'
        else 'Above Rolling Avg' end as performance_flag
from RollingAvgData
order by department, evaluation_date
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	department	evaluation_date	revenue_generated	rolling_avg_department_revenue	performance_flag
Jack	Finance	2024-02-20	40000	40000.00	Above Rolling Avg
Karen	Finance	2024-03-01	42000	41000.00	Above Rolling Avg
Leo	Finance	2024-03-05	38000	40000.00	Below Rolling Avg
Paul	Finance	2024-03-25	35000	38333.33	Below Rolling Avg
Tina	Finance	2024-04-10	36000	36333.33	Below Rolling Avg
Grace	HR	2024-02-01	10000	10000.00	Above Rolling Avg

Q7:

```sql
-- 7) Find Revenue Peaks Within a Dynamic Date Range
-- Business Need:
-- For each employee, calculate the maximum revenue achieved within a rolling 60-day
-- window (current row and preceding 60 days).

select
    emp_name,
    evaluation_date,
    revenue_generated,
    max(revenue_generated) over(order by evaluation_date range between interval '60' day preceding and current row) as max_rev_in_60_days
from employee_performance
where emp_name = 'quinn'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	evaluation_date	revenue_generated	max_rev_in_60_days
Quinn	2023-03-30	15000	15000
Quinn	2024-03-30	24000	24000
Quinn	2024-04-30	12000	24000

Q8:

```sql
-- 8) For each row for Quinn employee, make a flag to show that this revenue_generated is more
-- or less than the last revenue made by Quinn

select
    emp_name,
    project_id,
    evaluation_date,
    revenue_generated,
    case when revenue_generated > lag(revenue_generated) over (order by evaluation_date) then 'Increasing'
        else 'Not Increasing' end as trend_flag
from employee_performance
where emp_name = 'Quinn'
order by evaluation_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	project_id	evaluation_date	revenue_generated	trend_flag
Quinn	104	2023-03-30	15000	Not Increasing
Quinn	104	2024-03-30	24000	Increasing
Quinn	104	2024-04-30	12000	Not Increasing

Q9:

```sql
select
    customer_id,
    purchase_date,
    purchase_amount,
    lag(purchase_amount) over (partition by customer_id order by purchase_date) as previous_purchase,
    lead(purchase_amount) over (partition by customer_id order by purchase_date) as next_purchase,
    lag(purchase_date) over (partition by customer_id order by purchase_date) as previous_purchase_date,
    lead(purchase_date) over (partition by customer_id order by purchase_date) as next_purchase_date,
    purchase_date - lag(purchase_date) over (partition by customer_id order by purchase_date) as days_since_last_purchase,
    lead(purchase_date) over (partition by customer_id order by purchase_date) - purchase_date as days_until_next_purchase
from customer_purchases;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	purchase_date	purchase_amount	previous_purchase	next_purchase	previous_purchase_date	next_purchase_date	days_since_last_purchase	days_until_next_purchase
1	2024-01-01	100.00	NULL	150.00	NULL	2024-01-10	NULL	9
1	2024-01-10	150.00	100.00	200.00	2024-01-01	2024-01-15	9	5
1	2024-01-15	200.00	150.00	NULL	2024-01-10	NULL	5	NULL
2	2024-02-01	300.00	NULL	350.00	NULL	2024-02-10	NULL	9
2	2024-02-10	350.00	300.00	NULL	2024-02-01	NULL	9	NULL
3	2024-03-01	400.00	NULL	450.00	NULL	2024-03-05	NULL	4
3	2024-03-05	450.00	400.00	500.00	2024-03-01	2024-03-20	4	15
3	2024-03-20	500.00	450.00	NULL	2024-03-05	NULL	15	NULL